MODULE

26

WebSockets, background tasks & caching

Push live updates, move slow work off the request, and make hot data instant with Redis.

YOU WILL LEARN

WebSocket chat	`01_websockets_chat.py`
Background tasks	`02_background_tasks.py`
Caching with Redis	`03_caching_with_redis.py`
Interview Q&A	`INTERVIEW.md`

WHY THREE PERFORMANCE TOOLS

Three problems, three tools

THE PROBLEM

1) The dashboard should update **without refreshing**. 2) Sign-up takes 6 s because it **sends emails** in the request. 3) The product page runs the **same expensive query** 1,000 times a minute.

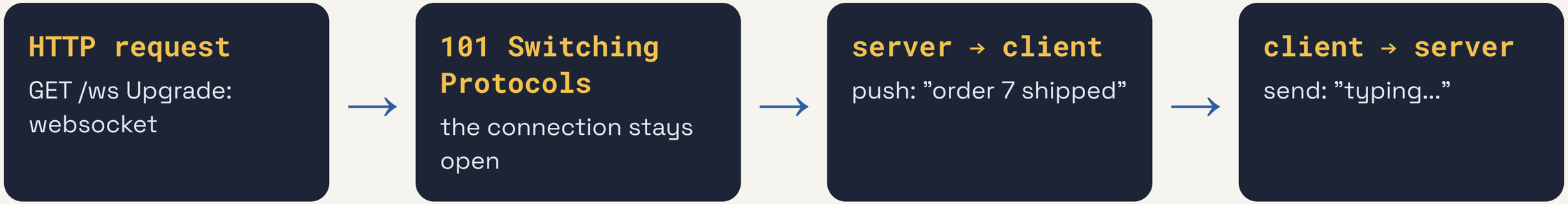
THE IDEA

1) WebSockets push updates. 2) Background jobs do slow work later. 3) A cache remembers hot results.

PICTURE IT WEBSOCKETS

HTTP vs WebSocket

📞 HTTP is like **sending letters**: the client asks, the server answers, done. A WebSocket is a **phone call**: the line stays open and **both** sides can talk at any time.



Use it for chat, live dashboards, notifications, multiplayer. Alternatives: polling, Server-Sent Events.

TRY IT WEBSOCKETS

A broadcast chat room in FastAPI

```python
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.testclient import TestClient

app = FastAPI()
clients: set[WebSocket] = set()

@app.websocket("/ws/chat")
async def chat(ws: WebSocket):
    await ws.accept()
    clients.add(ws)
    try:
        while True:
            msg = await ws.receive_text()
            for c in list(clients):            # broadcast to everyone
                await c.send_text(f"someone: {msg}")
    except WebSocketDisconnect:
        clients.discard(ws)

with TestClient(app).websocket_connect("/ws/chat") as ws:
    ws.send_text("hello room")
    print(ws.receive_text())
```

OUTPUT

```
someone: hello room
```

1. Accept, then **loop**: receive and broadcast.
2. Remove clients when they disconnect.
3. Several servers? Share messages via **Redis pub/sub**.

PICTURE IT BACKGROUND TASKS

Answer fast, work later

🍴 A restaurant waiter doesn't cook: he takes the order, gives you a **ticket number**, and the kitchen works on it. Your API returns **202 Accepted** + a job ID; a **worker** does the slow part.



TRY IT BACKGROUND TASKS

202 + polling with a simple job queue

```
import queue, threading, time, uuid
from fastapi import FastAPI
from fastapi.testclient import TestClient

app, jobs, q = FastAPI(), {}, queue.Queue()

def worker():
    while True:
        job_id = q.get()
        time.sleep(0.2)                          # the slow part
        jobs[job_id] = {"status": "done", "url": f"/files/{job_id}.pdf"}
threading.Thread(target=worker, daemon=True).start()
@app.post("/reports", status_code=202)
def create_report():
    job_id = uuid.uuid4().hex[:6]
    jobs[job_id] = {"status": "pending"}
    q.put(job_id)
    return {"job_id": job_id}
@app.get("/jobs/{job_id}")
def job_status(job_id: str): return jobs[job_id]
c = TestClient(app)
jid = c.post("/reports").json()["job_id"]
print(c.get(f"/jobs/{jid}").json()["status"])
time.sleep(0.4)
print(c.get(f"/jobs/{jid}").json()["status"])
```

OUTPUT

```
pending
done
```

1. In production the queue is **Redis/RabbitMQ** and the worker is **Celery** or RQ.
2. Small fire-and-forget jobs: FastAPI **BackgroundTasks**.
3. Jobs must be **idempotent**: they may run twice.

PICTURE IT CACHING

Cache-aside: the most common pattern

⚡ Check the fast cache first. On a **miss**, load from the slow DB and store the result with a **TTL** (expiry). When the data changes, **invalidate** the key.



TRY IT CACHING

Cache-aside with Redis (fakeredis here)

```python
import json, time
import fakeredis                        # same API as redis.Redis

r = fakeredis.FakeRedis()

def load_product_from_db(pid):
    time.sleep(0.05)                    # a slow query
    return {"id": pid, "name": "Lamp", "price": 49.0}

def get_product(pid):
    key = f"product:{pid}"
    if (cached := r.get(key)) is not None:
        return json.loads(cached), "hit"
    product = load_product_from_db(pid)
    r.set(key, json.dumps(product), ex=300)        # TTL 5 minutes
    return product, "miss"

for _ in range(3):
    t = time.perf_counter()
    _, how = get_product(7)
    print(f"{how:4} {(time.perf_counter() - t) * 1000:5.1f} ms")

r.delete("product:7")                   # after an update
print(get_product(7)[1], "after invalidation")
r.incr("views:7"); r.incr("views:7"); print("views:", int(r.get("views:7")))
```

OUTPUT

```
miss  55.8 ms
hit    0.1 ms
hit    0.1 ms
miss after invalidation
views: 2
```

1. **TTL** limits how stale data can get.
2. Invalidate on writes; ”there are only two hard things…”
3. Redis also does counters, rate limits, leaderboards, sessions.

COMMON MISTAKE CACHING

Caching per-user data under a shared key

✗ WRONG

```
key = "dashboard"              # same for everyone!
if cached := r.get(key):
    return cached              # user B sees user A's data
```

A cache key that ignores the **user** (or language, permissions) leaks private data between users.

✓ RIGHT

```
key = f"dashboard:user:{user.id}:v2"
if cached := r.get(key):
    return cached
```

Put **everything the result depends on** in the key; add a version suffix to invalidate all at once.

IN REAL JOBS REAL-TIME, TASKS, CACHING

In production systems

Celery task

```
@shared_task(bind=True, max_retries=3)
def send_invoice(self, order_id): ...
```

Rate limiting with Redis

```
n = r.incr(f"rl:{ip}:{minute}")
r.expire(...); if n > 100: raise 429
```

HTTP caching

```
Cache-Control: public, max-age=60
ETag + 304 Not Modified
```

Scheduled jobs

```
celery beat / cron: nightly reports,
cleanup of expired tokens
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How would you design caching for a slow endpoint?”

- Measure first: which query or call is slow and how often is it hit?
- Use **cache-aside** with Redis: get → on miss load and set with a **TTL**.
- Design the **key** from everything the result depends on (ids, user, language, version).
- **Invalidate** on writes (delete the key) or accept staleness up to the TTL.
- Watch for **stampedes** (many misses at once): locks, early refresh, jittered TTLs.

> **Follow-up they ask next:** ”Where can you cache?” (browser/CDN via HTTP headers, reverse proxy, app memory (lru_cache), Redis, DB query cache).

INTERVIEW SAY THIS IN THE INTERVIEW

“When do you use background jobs? WebSockets vs polling?”

- Background jobs: anything **slow or unreliable** that the user doesn’t need to wait for: emails, PDFs, image processing, calls to partner APIs.
- Return **202** + a job ID; workers (**Celery**/RQ) retry with backoff; tasks must be **idempotent**.
- **Polling** is simple but wasteful and laggy; **WebSockets** give instant two-way updates but need connection management and scaling (Redis pub/sub).
- **Server-Sent Events** fit one-way live feeds.

> **Follow-up they ask next:** ”What happens if a worker crashes mid-task?” (with acks_late the message is redelivered, so the task runs again: that’s why it must be idempotent).

RECAP REMEMBER THESE

Module 26 in 5 lines

1. WebSockets: an open, two-way connection; broadcast to rooms; Redis pub/sub to scale.
2. Slow work → queue + worker; return **202** and let the client poll or get pushed.
3. Jobs retry, so make them **idempotent**.
4. Cache-aside + **TTL** + invalidation; keys include everything the result depends on.
5. Redis: cache, counters, rate limits, queues, sessions.

PRACTICE NOW

Run 03_caching_with_redis.py, then add a cached GET /products/{id} that invalidates on PATCH.